

Шпаргалка по awk и sed

Две классические утилиты обработки текста: sed правит строки потоком, awk разбирает их на поля и считает. Примеры рассчитаны на GNU-версии; в BSD и macOS отличается запись -i.

sed: замена

```
sed 's/старое/новое/' файл
    Заменить первое вхождение в каждой строке

sed 's/старое/новое/g'
    Все вхождения в строке

sed 's/старое/новое/3'
    Только третье вхождение

sed 's/старое/новое/gI'
    Без учёта регистра

sed -i.bak 's/a/b/g' файл
    Правка на месте с сохранением копии файл.bak

sed 's|usr/local|opt|g'
    Другой разделитель — удобно для путей

sed -E 's/([0-9]+)-([0-9]+)/\2-\1/'
    Расширенные регулярные выражения и ссылки на группы

sed 's/.*/[&]/'
    & — весь найденный фрагмент
```

sed: выбор строк

```
sed -n '5p' файл
    Напечатать только пятую строку (-n гасит обычный вывод)

sed -n '10,20p'
    Диапазон строк

sed -n '/начало/,/конец/p'
    От строки с одним совпадением до строки с другим

sed '5d' / sed '/^#/d'
    Удалить пятую строку; удалить комментарии

sed '/^$/d'
    Убрать пустые строки

sed '$d' / sed -n '$p'
    Удалить последнюю строку; показать её

sed '/шаблон/s/a/b/g'
    Замена только в подходящих строках

sed '2i\текст' / '2a\текст'
    Вставить строку до и после второй

sed 'q5'
    Завершить с кодом возврата — удобно в сценариях
```

awk: устройство

Программа awk — это набор правил условие { действие }. Для каждой строки проверяются условия и выполняются подходящие действия. Строка делится на поля: \$1, \$2, ..., \$NF — последнее, \$0 — строка целиком.

```
awk '{print $1, $3}' файл
    Первое и третье поле

awk -F: '{print $1}' /etc/passwd
    Свой разделитель полей

awk -F'\t' -v OFS=';' '{print $1, $2}'
    Табуляция на входе, точка с запятой на выходе

awk 'NR==5' / awk 'NR>1'
    Пятая строка; пропустить заголовок

awk 'END {print NR}'
    Число строк

awk 'NF' файл
    Только непустые строки
```

awk: фильтры

```
awk '$3 > 100'
    Строки, где третье поле больше ста

awk '/ошибка/ {print $0}'
    По совпадению с шаблоном

awk '$1 ~ /^19/'
    Поле совпадает с выражением (!~ — не совпадает)

awk '$2 == "GET" && $9 >= 500'
    Условия объединяются

awk 'length($0) > 80'
    Слишком длинные строки

awk '!seen[$0]++'
    Удалить повторы, сохранив исходный порядок
```

awk: подсчёты

```
# Сумма и среднее по третьему полю
awk '{sum += $3} END {print sum, sum/NR}' файл

# Количество по ключу
awk '{count[$1]++} END {for (k in count) print count[k], k}' файл |
    sort -rn | head

# Сумма по группе
awk -F, '{total[$2] += $3} END {for (g in total) print g, total[g]}' data.csv

# Максимум
awk 'NR==1 || $2 > max {max = $2; line = $0} END {print line}' файл
```

awk: блоки и функции

```
BEGIN { ... }  
    Выполняется до чтения данных: заголовки, настройка  
    разделителей  
  
END { ... }  
    После всех строк: итоги  
  
printf "%-10s %5.2f\n", $1, $2  
    Форматированный вывод, как в C  
  
substr($0, 1, 10)  
    Подстрока  
  
split($0, a, ",")  
    Разбить строку в массив  
  
gsub(/старое/, "новое")  
    Замена внутри awk; sub — только первое вхождение  
  
toupper($1) / tolower($1)  
    Регистр  
  
next / exit  
    Перейти к следующей строке; завершить работу
```

Готовые рецепты

```
# Топ-10 адресов в журнале nginx  
awk '{print $1}' access.log | sort | uniq -c | sort -rn  
| head  
  
# Коды ответа и их доля  
awk '{c[$9]++} END {for (s in c) printf "%s %d\n", s, c[s], c[s]*100/NR}' access.log  
  
# Средняя длительность запроса  
awk '{s += $NF} END {print s/NR}' timings.txt  
  
# Сумма третьего столбца CSV без заголовка  
awk -F, 'NR>1 {s += $3} END {print s}' data.csv  
  
# Убрать пробелы по краям строк  
sed -E 's/^[[:space:]]+|[[:space:]]+$//g' файл  
  
# Комментировать строку в конфигурации  
sed -i '/^Port /s/^/#/' /etc/ssh/sshd_config  
  
# Вывести значения без комментариев и пустых строк  
sed -E '/^\s*(#|$)/d' config.conf
```

Что помнить

- Проверяйте команду без `-i`, глядя на вывод, и только потом правьте файл на месте.
- В GNU `sed -i` работает без аргумента, в BSD и macOS требуется суффикс: `-i ''`.
- Спецсимволы регулярных выражений в заменяемом тексте нужно экранировать: точка, слеш, звёздочка.
- Одинарные кавычки вокруг программы защищают \$1 от подстановки оболочкой.
- Для JSON берите `jq`, для CSV с кавычками — специализированные утилиты: `awk` не знает про экранирование внутри полей.
- Если программа `awk` выросла за десяток строк, вынесите её в файл и запускайте `awk -f prog.awk`.